



Master Auth Guards in Angular

Protect routes. Control access. Like a pro.

Niyati Kaneriya



Why You Need Auth Guards ?



Would you let anyone into your house?

No? Then don't let anyone into your app routes.



Auth Guards = Route Bouncers



They check:

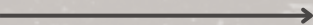
- Is the user allowed?
- Redirect if not!

Meet the Guard Squad

Angular gives you 5 types of guards:

Guard	What it does
CanActivate	Route access gate
CanActivateChild	Child routes gate
CanDeactivate	Warn before leaving
CanLoad	Blocks lazy-loaded modules
Resolve	Pre-fetch data before route

Use the right guard, not just any.



Build Your First AuthGuard

```
1 @Injectable({ providedIn: 'root' })
2 export class AuthGuard implements CanActivate {
3     constructor(private auth: AuthService, private router: Router) {}
4
5     canActivate(): boolean {
6         return this.auth.isLoggedIn() || this.router.parseUrl('/login');
7     }
8 }
```

Logic lives in the guard,
Block users before they even hit
the page

Plug It In

```
1 {  
2   path: 'dashboard',  
3   component: DashboardComponent,  
4   canActivate: [AuthGuard]  
5 }
```

- Route protected
 - If not logged in, user gets rerouted
- No loopholes. Clean access control.

Stop Lazy Loading Too

Don't want users to even download the module?

Use CanLoad:



```
1  canLoad(): boolean {  
2    return this.auth.isLoggedIn();  
3  }
```

Guard the bundle before it loads!

Pro Tips for Auth Guards

- Use services for clean auth logic
- Combine with role checks for RBAC
- Guards work before route renders
- Chain multiple guards if needed!

TL;DR

Auth Guards =

- Safer apps
- Better UX
- Cleaner route logic

Start using them — your users & dev team will thank you.

